Foundations of cybersecurity

Cryptographic Timestamping Service

ISAIA PORCU - 635930, Federico Volpi - 615725,

Index

1 Introduction

This project implements a **Cryptographic Timestamping Service**. A user wants to prove that a certain document already existed at a certain moment in time, without sending the whole document to the server.

Instead of uploading the file, the user computes its cryptographic hash and sends only that hash to the service. The server, which plays the role of a trusted Time Stamping Authority (TSA), attaches the current UTC time to the hash and signs the result. The signed result is the timestamp token.

Later, anyone can verify the token by checking the signature over the same document hash and timestamp. If the verification succeeds, then the hash and timestamp have not been modified and the token was produced by the TSA private signing key.

The service therefore answers the question:

“Can I prove that this exact document existed before this time?”

The answer is yes, as long as the verifier trusts the TSA signing key and the submitted hash really belongs to the document being claimed.

1.1 Main actors

The system has three components:

1. **Client** - *Client/Client.py*

   The command-line program used by the user. It connects to the server, authenticates, asks for the account balance, requests new timestamps, and verifies existing timestamp tokens.

2. **TSA Server** - *Server/Server.py*

   The TCP server listening on port *1488*. It authenticates itself to the client, creates a protected communication channel, checks user credentials, manages timestamp quotas, signs timestamp tokens, and verifies signatures.

3. **Database simulator** - *Server/Database.py*

   A JSON-based persistent storage component. It stores registered users, bcrypt password hashes, and two counters: *available* timestamps and *used* timestamps.

1.2 What a timestamp token contains

A successful timestamp response contains three important fields:

- the document hash, encoded in hexadecimal;
- the UTC timestamp generated by the server;
- an RSA-PSS signature over the binary hash concatenated with the timestamp bytes.

The server signs:

$$\text{hash} \parallel \text{timestamp}_{\text{bytes}}$$

If an attacker changes the hash or the timestamp, the signature verification fails.

2 Security Goals

The service is designed around four practical security goals.

2.1 Confidentiality

After the handshake, all client-server messages are encrypted. An attacker observing the network should not be able to read usernames, passwords, requested hashes, balances, signatures, or verification requests. This is provided by **ChaCha20Poly1305**, an authenticated encryption scheme.

2.2 Integrity

Messages must not be silently changed in transit. If an attacker modifies an encrypted command, the receiver should reject it. ChaCha20Poly1305 also provides this property because each ciphertext includes an authentication tag. If the ciphertext is altered, decryption fails.

2.3 Server authenticity

The client must know that it is talking to the real TSA server, not to a machine placed in the middle of the connection.

The server owns a long-term RSA-4096 key pair used for connection authentication:

$$(\mathrm{pubK_c}, \mathrm{privK_c})$$

The public key $\mathrm{pubK_c}$ is bound to a server certificate. The client already has the authentic certificate before the connection starts and uses it as a local trust anchor. During the handshake, the server signs the handshake transcript with $\mathrm{privK_c}$. The client extracts $\mathrm{pubK_c}$ from the certificate and verifies the signature with that key.

This prevents a Man-in-the-Middle attacker from replacing the server key exchange values, because the attacker cannot create a valid signature without the server private key.

2.4 Perfect Forward Secrecy

If a long-term server key is compromised in the future, old encrypted sessions should still remain protected.

For this reason, the session key is not encrypted with RSA and sent across the network. Instead, the client and server generate fresh **X25519 ephemeral keys** for each connection and derive a shared secret. These ephemeral private keys are used only for that session.

The shared secret is then processed with **HKDF-SHA256** to obtain the final 32-byte session key used by ChaCha20Poly1305.

3 Cryptographic Design

This section explains each primitive by its role in the system.

3.1 Secure channel setup

Before the client can send credentials or timestamp requests, the two sides establish a secure channel.

1. The client loads the trusted server certificate and extracts the certified connection-authentication public key.
2. The client generates a fresh X25519 ephemeral key pair and a 32-byte random nonce.
3. The client sends its public key and nonce to the server.
4. The server generates its own fresh X25519 ephemeral key pair and a 32-byte random nonce.
5. The server signs the complete handshake transcript with its RSA connection-authentication private key.
6. The client verifies the signature using the public key extracted from the trusted server certificate.
7. Both sides compute the same X25519 shared secret.
8. Both sides derive the same symmetric session key with HKDF-SHA256.
9. From this point on, application messages are encrypted with ChaCha20Poly1305.

The transcript signed by the server is:

$$\text{client\_pub} \parallel \text{client\_nonce} \parallel \text{server\_pub} \parallel \text{server\_nonce}$$

Signing the whole transcript is important because it binds together both parties' public key exchange values and both random nonces. The client is not only checking that the server signed something; it is checking that the server signed the exact handshake that the client participated in.

3.2 Key derivation

The raw X25519 shared secret is not used directly as an encryption key. Both sides run it through HKDF-SHA256:

- *input key material*: the X25519 shared secret;
- *salt*: *client_nonce* + *server_nonce*;
- *info*: *b"session encryption"*;
- *output*: a 32-byte session key.

The salt makes the derived key depend on fresh randomness from both sides.

3.3 Encrypted messages

After the handshake, every application message is a JSON object encrypted with ChaCha20Poly1305. The encrypted payload contains:

- a 12-byte ChaCha20Poly1305 nonce;
- the ciphertext, which also includes the authentication tag.

This means the protocol protects both secrecy and tamper detection for post-handshake messages.

3.4 Timestamp signing

The service uses a second RSA-4096 key pair for timestamp tokens:

$$(\text{pubK}_{\text{ts}}, \text{privK}_{\text{ts}})$$

This key pair is separate from the connection-authentication key pair.

When a timestamp is requested, the server signs:

$$\text{hash} \parallel \text{timestamp}_{\text{bytes}}$$

using **RSA-PSS** padding and **SHA256**. RSA-PSS is used instead of older deterministic RSA signature padding because it is the modern randomized padding scheme recommended for RSA signatures.

3.5 Replay protection

Encryption alone does not automatically stop replay attacks. An attacker may not be able to read an encrypted message, but might still record it and send the same bytes again.

This is especially relevant for timestamp requests, because replaying a previously valid request could consume a user's quota or repeat an operation.

To reduce this risk, the server maintains a monotonic replay counter after login:

1. After a successful login, the server initializes the expected counter to 0.
2. The server sends this value to the client inside the encrypted login response.
3. Every later client request must include the current replay counter in the JSON body.
4. The server checks the received value against the expected one.
5. If it matches, the server increments the counter and returns the new value in the response.
6. If it does not match, the server treats the message as suspicious and closes the connection.

Each counter value is valid only once. A captured old request contains an old counter, so it will not match the server's current expected value.

4 Message Formats

This section describes what is actually sent on the network.

4.1 Framing

After the initial raw handshake, messages use a length-prefixed format:

1. **Header**: 4 bytes, big-endian unsigned integer (*>I*), containing the payload length.
2. **Payload**: the message bytes.

For encrypted application messages, the payload is:

1. **Encryption nonce**: 12 bytes, used by ChaCha20Poly1305.
2. **Ciphertext**: encrypted JSON plus authentication tag.

This framing lets the receiver know exactly how many bytes belong to the next message.

4.2 Plaintext handshake

The first handshake values are not encrypted yet, because their purpose is to create the encrypted channel.

1. **Client hello** - Client → Server
   - *Size*: 64 bytes.
   - *Format*: *client_pub_bytes* (32 bytes) followed by *client_nonce* (32 bytes).
2. **Server hello** - Server → Client
   - *Size*: 576 bytes.
   - *Format*: *server_pub_bytes* (32 bytes), *server_nonce* (32 bytes), and *signature* (512 bytes).
   - *Signature input*: *client_pub_bytes* ‖ *client_nonce* ‖ *server_pub_bytes* ‖ *server_nonce*.
3. **Handshake confirmation** - Server → Client
   - *Format*: length-prefixed message.
   - *Payload*: *b"Handshake successful"*.

4.3 Login

The login request is the first encrypted JSON message.

- Client → Server:

```
{
  "username": "<username_string>",
  "password": "<password_string>"
}
```

- Server → Client:

```
{
  "status": "success",
  "counter": 0
}
```

If login fails, the server closes the connection instead of continuing the session.

4.4 Balance check

- Client → Server:

```
{
  "request": "balance",
  "counter": <current_replay_counter>
}
```

- Server → Client:

```
{
  "available": <integer_remaining_tokens>,
  "used": <integer_consumed_tokens>,
  "counter": <new_replay_counter>
}
```

4.5 Timestamp request

- Client → Server:

```
{
  "request": "timestamp",
  "hash": "<hex_encoded_document_hash>",
  "counter": <current_replay_counter>
}
```

- Server → Client, when the user still has available timestamps:

```
{
  "hash": "<hex_encoded_document_hash>",
  "timestamp": "<UTC_time_string>",
  "signature": "<hex_encoded_rsa_pss_signature>",
  "counter": <new_replay_counter>
}
```

- Server → Client, when the quota is exhausted:

```
{
  "status": "failed",
  "message": "Uses exhausted!",
  "counter": <new_replay_counter>
}
```

4.6 Timestamp verification

- Client → Server:

```
{
  "request": "verify",
  "hash": "<hex_encoded_document_hash>",
  "timestamp": "<UTC_time_string>",
  "signature": "<hex_encoded_rsa_pss_signature>",
  "counter": <current_replay_counter>
}
```

- Server → Client, valid token:

```
{
  "status": "success",
  "valid": true,
  "message": "Timestamp is valid!",
  "counter": <new_replay_counter>
}
```

- Server → Client, invalid or modified token:

```
{
  "status": "success",
  "valid": false,
  "message": "Invalid timestamp or altered data!",
  "counter": <new_replay_counter>
}
```

4.7 Quit

- Client → Server:

```
{
  "request": "quit",
  "counter": <current_replay_counter>
}
```

After this request, the socket is closed.

5 Password Storage

The database simulator never stores plaintext passwords. Each user entry stores a bcrypt hash string plus the timestamp quota counters.


```
"Giacomo": {
  "password": "$2b$12$BKrwYnXiEXfTQHz2reiiN.EHlvWOpnDlCznSo3sxg.AubUYTmBGee",
  "available": 0,
  "used": 5
}
```


Bcrypt is appropriate here because it is intentionally slow compared with ordinary hash functions such as SHA256. If an attacker steals the JSON database, they cannot directly read the passwords. They must guess candidate passwords and run bcrypt for each guess.

5.1 Bcrypt string format

A bcrypt hash includes the algorithm version, cost parameter, salt, and final hash in one string.

$$\text{Format: \$< Variant >\$< Cost >\$< Salt and Hash >}$$

Example:

$2b$12$BKrwYnXiEXfTQHz2reiiN.EHlvWOpnDlCznSo3sxg.AubUYTmBGee

Meaning:

- *$2b$* identifies the bcrypt variant.
- *12* is the work factor. It means bcrypt uses $2^{12} = 4096$ internal rounds.
- *BKrwYnXiEXfTQHz2reiiN.* is the encoded salt.
- *EHlvWOpnDlCznSo3sxg.AubUYTmBGee* is the encoded hash output.

The salt is generated randomly when a user is registered. Therefore, two users with the same password should still have different stored bcrypt strings.

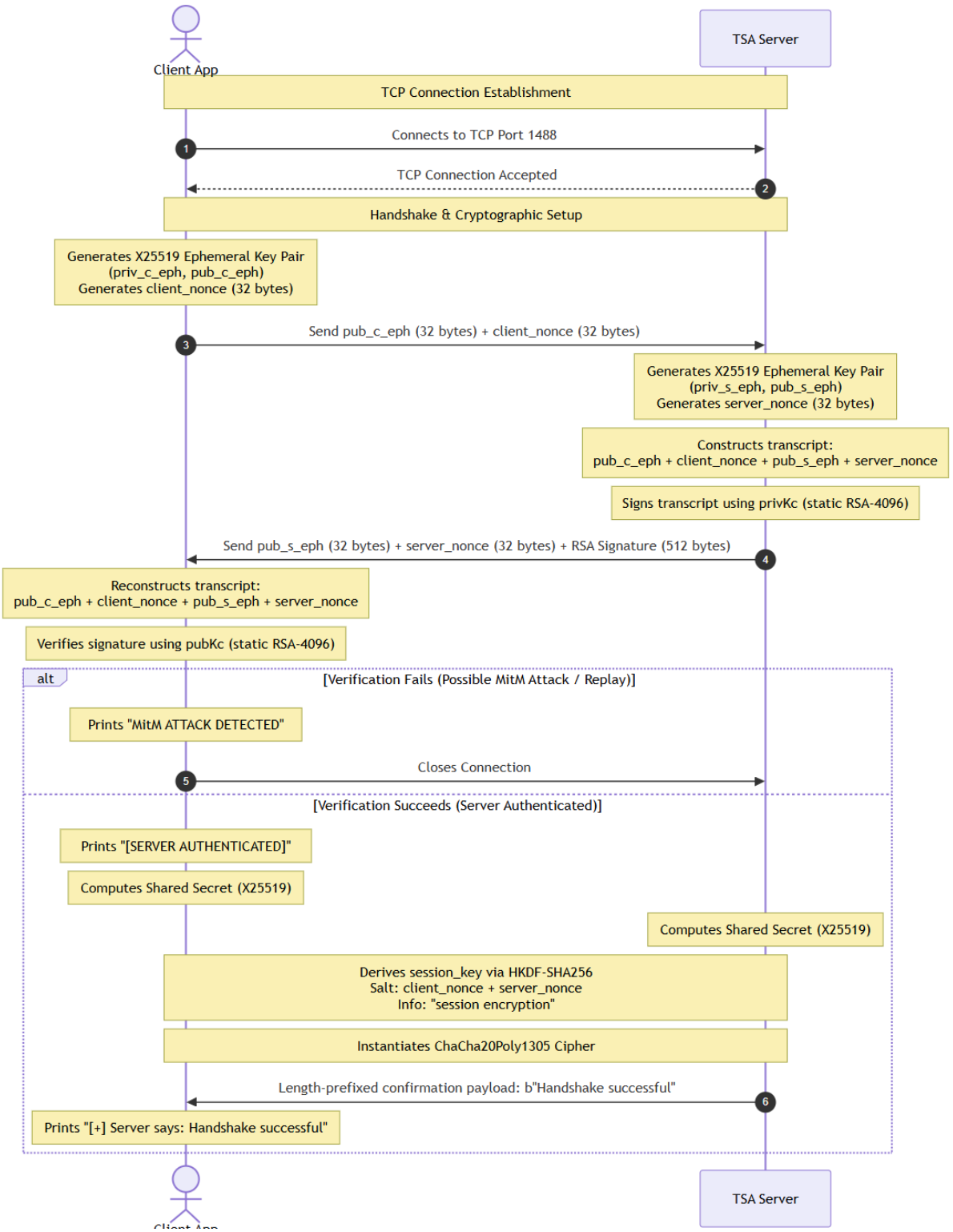
During login, the server calls *bcrypt.checkpw*. This function reads the cost and salt from the stored bcrypt string, hashes the submitted password with the same parameters, and reports whether the result matches.

6 Protocol Walkthrough

The following diagrams show the protocol as a sequence of steps. The important thing to notice is the order:

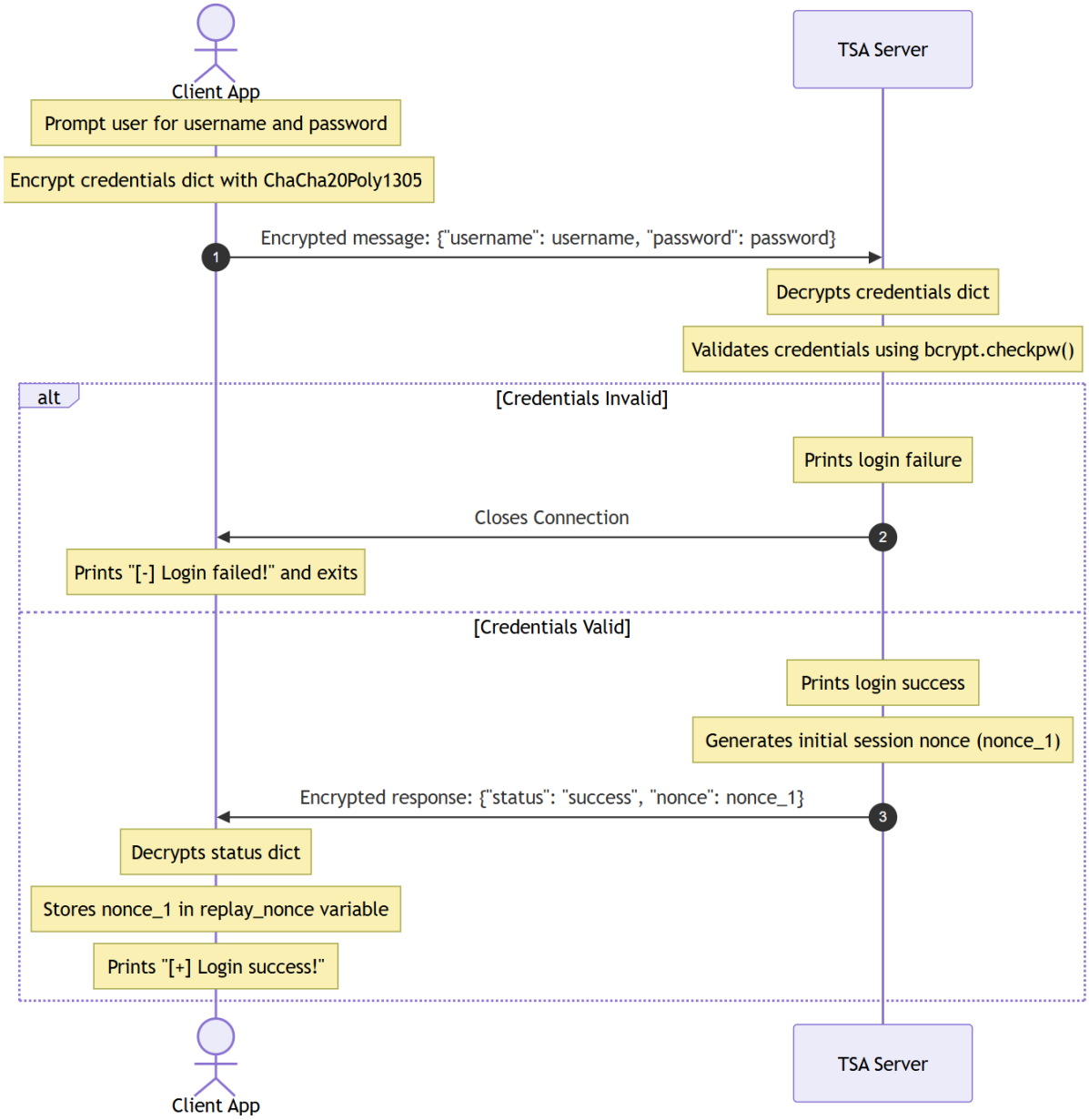
6.1 Handshake

The handshake establishes the encrypted channel and authenticates the server.



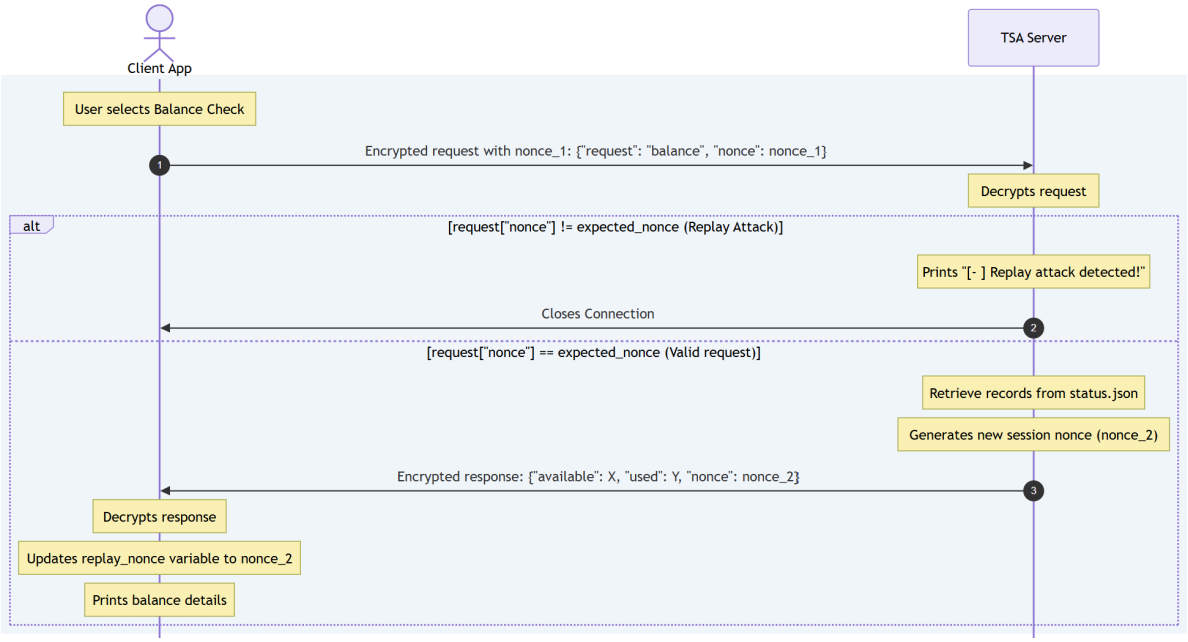
6.2 Login

After the channel is encrypted, the client sends username and password. If the credentials are valid, the server sends the first replay counter.



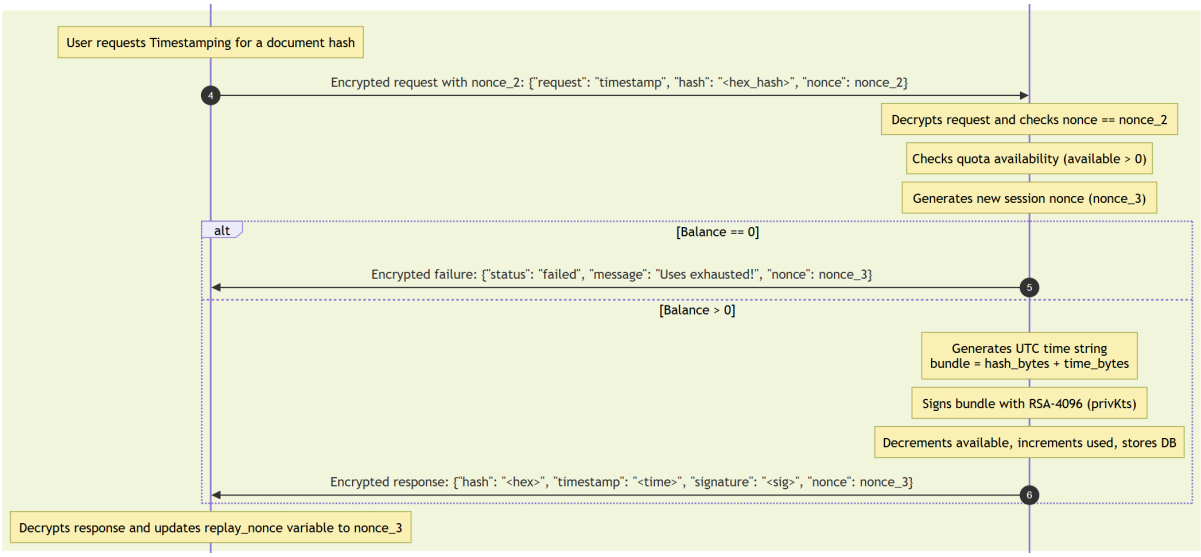
6.3 Balance

The balance request shows how ordinary session commands work: the client sends the current replay counter, and the server returns a new one.



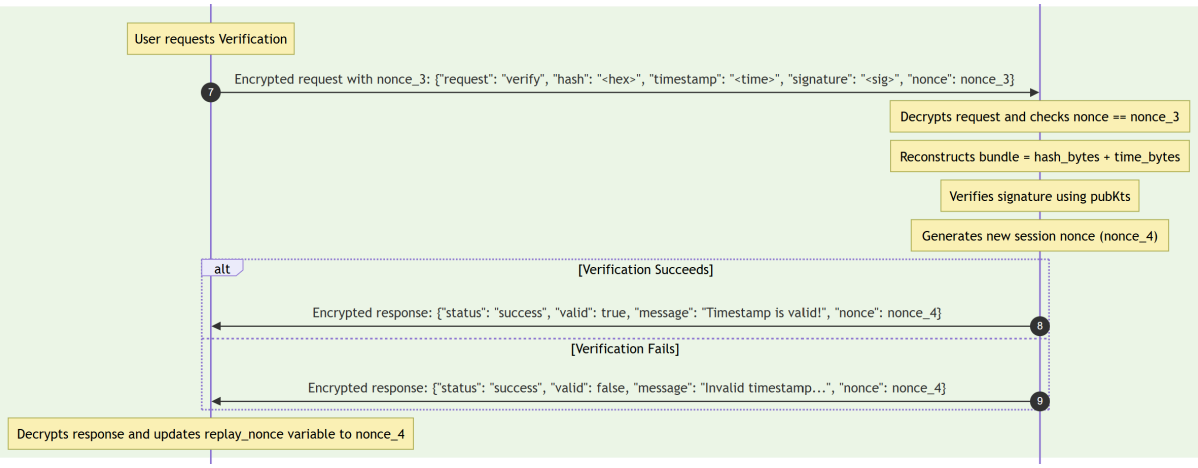
6.4 Timestamping

For timestamping, the client sends a document hash. The server checks the user's quota, generates the current UTC time, signs the hash and timestamp together, decreases the quota, and returns the token.



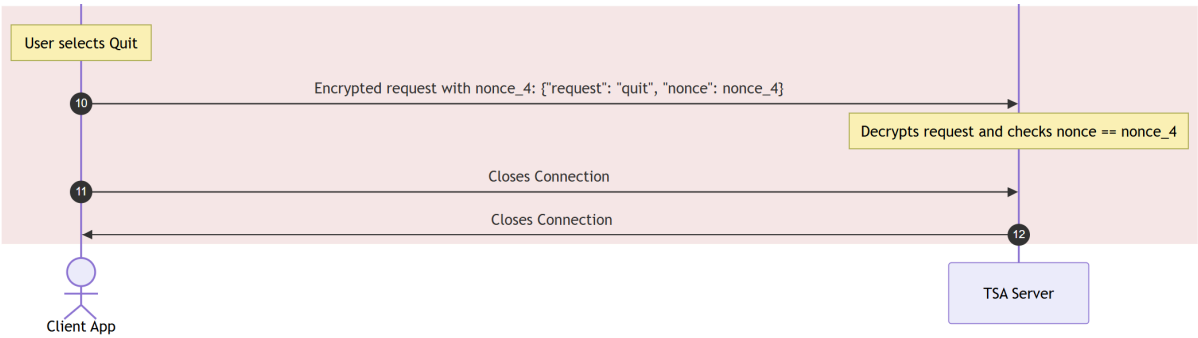
6.5 Verification

For verification, the server reconstructs the same bundle, hash $\|$ $\text{timestamp}_{\text{bytes}}$, and verifies the RSA-PSS signature with pubK_{ts}.



6.6 Quit

The quit request closes the session.



7 Demo Logs

The following examples show the service behavior from the client point of view.

7.1 Successful timestamp

The user chooses option *2*, submits a document hash, and receives a timestamp token.

```
Welcome! What do you want to do?
0 - See my balance.
1 - Verify timestamp.
2 - Timestamp an hash.
3 - Quit.
Send request n. 2
Inserisci l'hash del documento da firmare (in formato HEX):
7b5cb4e5a9757657158434771605ec10e30d1d29fc2f0e0f3be8f45a278917e3
Working on timestamping, please wait...
[+] Server replied with:
{
   "hash": "7b5cb4e5a9757657158434771605ec10e30d1d29fc2f0e0f3be8f45a278917e3",
   "timestamp": "2026-06-21T19:02:54Z",
   "signature": "15d49ddf5e8d43bdecca5a9b037db49b48bad4995e75ea3dfa7528135dc4bdf90e...",
   "counter": 1
}
```

The returned signature is long because it is an RSA-4096 signature encoded in hexadecimal. The returned counter must be used in the next request.

7.2 Timestamp verification

The user chooses option *1* and provides the hash, timestamp, and signature. The server verifies that the signature matches the hash and timestamp.

```
Welcome! What do you want to do?
0 - See my balance.
1 - Verify timestamp.
2 - Timestamp an hash.
3 - Quit.
Send request n. 1
Inserisci l'hash del documento (in formato HEX): 7b5cb4e5a9757657158434771605ec10e30d1d29fc2f0e0f3be8f45a278917e3
Inserisci il timestamp (es. 2026-06-21T14:45:30Z): 2026-06-21T19:02:54Z
Inserisci la firma (in formato HEX): 15d49ddf5e8d43bdecca5a9b037db49b48bad4995e75ea3dfa7528135dc4bdf90e73939...
Working on verification, please wait...
[+] Server replied with:
{
   "status": "success",
   "valid": true,
   "message": "Timestamp is valid!",
   "counter": 2
}
```

The field *valid: true* means that the signature verification succeeded.

7.3 Unsuccessful timestamp

In this example the user has no remaining timestamp credits. The balance shows *available: 0*, so the next timestamp request fails.

```
[+] Connesso al server TSA su 127.0.0.1:1488
[+] Connessione stabilita con server
[>] Sending effimerate key to Server.
[+] [SERVER AUTHENTICATED]
[+] Canale sicuro stabilito (PFS abilitato).
[+] Server says: Handshake successful
Please insert your username:
```

```
Isaia
Please insert your password:
Isaia
[+] Login success!
Welcome! What do you want to do?
0 - See my balance.
1 - Verify timestamp.
2 - Timestamp an hash.
3 - Quit.
Send request n. 0
[+] Server replied with:
{
    "available": 0,
    "used": 5,
    "counter": 1
}
Welcome! What do you want to do?
0 - See my balance.
1 - Verify timestamp.
2 - Timestamp an hash.
3 - Quit.
Send request n. 2
Inserisci l'hash del documento da firmare (in formato HEX):
7b5cb4e5a9757657158434771605ec10e30d1d29fc2f0e0f3be8f45a278917e3
Working on timestamping, please wait...
[+] Server replied with:
{
    "status": "failed",
    "message": "Uses exhausted!",
    "counter": 2
}
```

8 Limitations and Assumptions

The implementation demonstrates the main cryptographic ideas, but it also relies on several assumptions.

1. **Trusted server certificate distribution**

   The client already has the authentic server certificate file, which contains pubK_c and acts as the local trust anchor. In a real deployment, this certificate would need to be distributed through a trusted installation process, trusted configuration, or CA-based mechanism.

2. **Local demonstration environment**

   The server listens on *127.0.0.1:1488*. This is appropriate for testing and demonstration, but a real network deployment would require operational hardening.

3. **JSON database simulator**

   The database is a simple JSON file. This is clear for a university project, but it is not a production database with concurrency control, audit logs, backups, or access control.

4. **Timestamp trust**

   The timestamp is generated from the server system clock. Therefore, the correctness of the timestamp depends on the server clock being accurate and trusted.

5. **Hash responsibility**

   The server signs the hash it receives. It does not see the original document, so the user is responsible for computing the hash correctly and preserving the document.

9 Conclusion

The project implements a complete educational timestamping service: it establishes an authenticated encrypted channel, protects user credentials, prevents simple replay of session commands, signs timestamp tokens, and verifies them later.

The most important design idea is separation of responsibilities. X25519 and HKDF create fresh session keys, ChaCha20Poly1305 protects the communication, RSA-PSS authenticates the server and signs timestamp tokens, bcrypt protects stored passwords, and monotonic replay counters make captured requests unusable after they have been processed once.

In short, the service lets a user prove that a document hash existed at a specific UTC time, while also showing how several cryptographic primitives cooperate inside a realistic client-server protocol.